



Casa abierta al tiempo

Universidad Autónoma Metropolitana

Unidad Iztapalapa

División de Ciencias Básicas e Ingeniería

Licenciatura en Computación

Proyecto Terminal de Investigación I

Informe Final — Semana 6

Detección Inteligente de Paráfrasis mediante Modelos de Lenguaje de Gran Escala (LLM)

Corpus humanos de paráfrasis, back-translation con MarianMT, clasificación de tipos, fundamentos de BERT/BETO y escalamiento del sistema al corpus completo

Asesor: Dr. Benjamin Moreno Montiel

Alumno: Juan Antonio Soria Rodríguez

Matrícula: 2203041232

Grupo: CK06

Fecha: Junio 2026

Índice

1. Resumen ejecutivo	2
2. Tarea 1: Corpus de paráfrasis humanos y ratificados	3
2.1. Qué busqué y por qué	3
2.2. Corpus que encontré	3
2.3. Prueba: ¿mi modelo generaliza a un corpus humano externo?	3
3. Tarea 2: MarianMT y back-translation en cascada	4
3.1. Qué exploré	4
3.2. Cómo lo implementé	4
3.3. Resultado	4
4. Tarea 3: ¿El modelo aprende el enfoque? Clasificación de tipos	5
4.1. La pregunta	5
4.2. Los tipos que distingue	6
4.3. Qué tipos aparecen realmente en el corpus	6
5. Tarea 4: BLEU, BERT y BETO — fundamentos	6
5.1. ¿Qué es el BLEU score?	7
5.2. BLEU contra la representación semántica de BERT	7
5.3. ¿Qué es BERT?	7
5.4. ¿Qué es BETO?	8
5.5. Tokenización y representación vectorial	8
6. Tarea 5: Escalar el sistema al corpus completo	9
6.1. El problema que diagnosticué	9
6.2. La estrategia de escalamiento	9
7. Sistemas desplegados implementados (v2 y v3)	11
7.1. Sistema v2: el pipeline base	11
7.2. Sistema v3: tres capas y árbitro LLM	11
7.3. Desempeño medido	12
8. Limitantes actuales	12
9. Próximos pasos: propuesta del sistema v4	13
10. Conclusiones	14
Glosario	15

1. Resumen ejecutivo

En esta semana atendí las cinco tareas que mi asesor me asignó, orientadas a fortalecer los fundamentos del proyecto y a preparar el escalamiento del sistema al corpus completo de tesis de la División de Ciencias Básicas e Ingeniería (CBI). Investigué la existencia de corpus de paráfrasis humanos y ratificados; exploré la generación de paráfrasis con *back-translation* en cascada usando MarianMT; verifiqué si mi modelo aprende a distinguir el *enfoque* o tipo de paráfrasis; profundicé en los fundamentos de BLEU, BERT y BETO (tokenización y representación vectorial); y diseñé la estrategia para incorporar todo el corpus al sistema.

Adicionalmente, documento en detalle los dos sistemas desplegables que implementé esta semana (v2 y v3), explico cómo funcionan y por qué tomé cada decisión de diseño, expongo sus limitantes actuales medidas con pruebas, y propongo un plan de mejora concreto: el sistema v4.

Cuadro 1: Respuesta directa a las cinco tareas asignadas por el asesor.

#	Tarea	Resultado
1	Corpus de paráfrasis humanos y ratificados; ¿podemos usarlos?	Existen (PAWS-X, MRPC, QQP) pero son en inglés o de dominio general. El más útil es mi propio gold standard humano ($\kappa = 0,957$). PAWS-X sirve como tarea intermedia.
2	MarianMT con traducción en cascada para generar paráfrasis	Lo implementé y evalué. Conclusión: no sirve como dato de entrenamiento — introduce calcos sintácticos y degrada el modelo.
3	¿El modelo aprende el enfoque? Clasificar tipos de paráfrasis	Sí. El sistema clasifica en 5 (v2) o 7 (v3) tipos combinando semántica (BETO) y solapamiento léxico (BLEU).
4	Entender BLEU; qué son BERT y BETO, tokenización y vectores	Documentado de forma didáctica con diagramas, distribuciones de BLEU y la comparación BLEU vs. BETO.
5	Meter todo el corpus; escalar el sistema	Diagnosticué que solo se indexa el 2 % del corpus; diseñé el escalamiento (índice completo + FAISS IVF + recuperación en dos etapas).

2. Tarea 1: Corpus de paráfrasis humanos y ratificados

2.1. Qué busqué y por qué

Mi asesor me pidió investigar si existen corpus de paráfrasis *ratificados por humanos* y verificar si puedo usarlos para mi modelo y mi sistema. La motivación es clara: a lo largo del proyecto demostré que los datos sintéticos (generados por LLM o por traducción automática) no sirven para entrenar un detector de paráfrasis humana, porque el modelo aprende el estilo del generador y no la relación semántica. Por eso necesito datos anotados por personas.

2.2. Corpus que encontré

Cuadro 2: Corpus de paráfrasis ratificados por humanos revisados.

Corpus	Idioma	Anotación	¿Sirve para mi proyecto?
PAWS-X	Español	Humana, adversarial	Parcial: es de dominio general, pero sirve como <i>tarea intermedia</i> .
MRPC	Inglés	Humana (Microsoft)	No directamente: idioma y dominio distintos.
QQP	Inglés	Humana (Quora)	No directamente: preguntas, no texto académico.
Mi gold standard	Español	Humana ($\kappa = 0,957$)	Sí : es del dominio CBI, anotado por mí, con 4 niveles de dificultad.

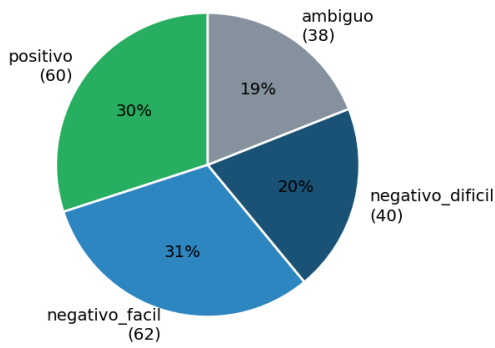
PAWS-X (*Paraphrase Adversaries from Word Scrambling*, versión multilingüe) es el corpus humano más relevante en español. Sus pares negativos son adversariales: dos frases con las mismas palabras pero distinto significado, lo que obliga al modelo a entender la estructura y no solo el vocabulario. MRPC y QQP son los corpus de referencia del *benchmark* GLUE, pero están en inglés.

Mi propio **gold standard** es el activo más valioso: 200 pares del corpus CBI anotados a mano, con un acuerdo inter-anotador de $\kappa = 0,957$ (casi perfecto según la escala de Landis y Koch). La Figura 1 muestra su composición por nivel de dificultad y la confianza del anotador.

2.3. Prueba: ¿mi modelo generaliza a un corpus humano externo?

Para verificar si podía usar estos corpus, evalué mi modelo BETO sobre PAWS-X en español. El resultado fue contundente: AUROC = 0,526 (casi azar) y una tasa de verdaderos positivos al 1 % de falsos positivos de apenas 0.011. Es decir, **mi modelo especializado en el dominio CBI no transfiere a PAWS-X**. Esto confirma un fenómeno de *domain shift*: las paráfrasis de Wikipedia (PAWS-X) son lingüísticamente distintas a las de tesis académicas.

Gold standard: 200 pares por nivel de dificultad



Confianza del anotador humano

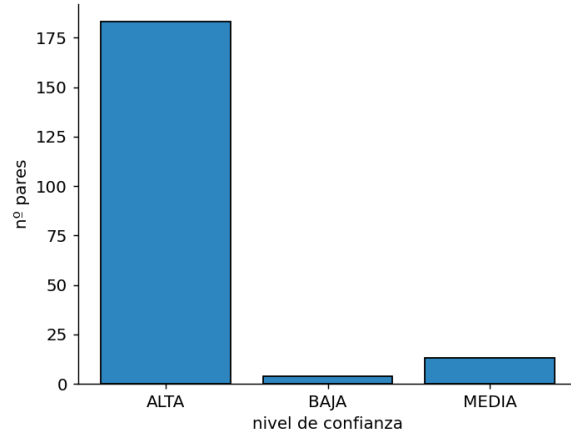


Figura 1: Composición de mi gold standard humano de 200 pares: distribución por nivel de dificultad (izquierda) y confianza del anotador (derecha). Los niveles *negativo difícil* y *ambiguo* son los que separan un detector semántico de una simple búsqueda léxica.

Conclusión de la Tarea 1

Los corpus humanos ratificados existen, pero los públicos (MRPC, QQP) están en inglés y PAWS-X es de dominio general. **No puedo usarlos como datos de entrenamiento directo** porque mi modelo no generaliza entre dominios. Sin embargo, **sí puedo aprovechar PAWS-X como *tarea intermedia***: entrenar primero en PAWS-X y después en mi gold standard del dominio CBI (técnica que detallo en los próximos pasos). El corpus humano que sí sirve directamente es el mío, que amplíé esta semana con 11 paráfrasis escritas manualmente.

3. Tarea 2: MarianMT y back-translation en cascada

3.1. Qué exploré

Mi asesor me pidió explorar la metodología de *back-translation* (retro-traducción) en cascada con MarianMT para generar paráfrasis, y verificar si puedo usarla para mi modelo. La idea es atractiva: si traduzco una frase del español al inglés y de vuelta al español, obtengo una reformulación automática que podría servir como paráfrasis sintética para aumentar mis datos.

3.2. Cómo lo implementé

Usé los modelos MarianMT de Helsinki-NLP (*opus-mt-es-en* y *opus-mt-en-es*) para construir una cascada *español* $\rightarrow$ *inglés* $\rightarrow$ *español*. Procesé cientos de fragmentos del corpus y generé dos colecciones: un corpus mixto (back-translation + paráfrasis de LLM) y un corpus de back-translation puro. Con ellos entrené dos versiones de mi modelo: BETO v6 (mixto) y BETO v7 (back-translation puro).

3.3. Resultado

La back-translation **empeoró el modelo**. BETO v7 (back-translation puro) obtuvo $F1 = 0,643$, muy por debajo de la v3 ($F1 = 0,842$) entrenada con datos humanos. La razón es que MarianMT

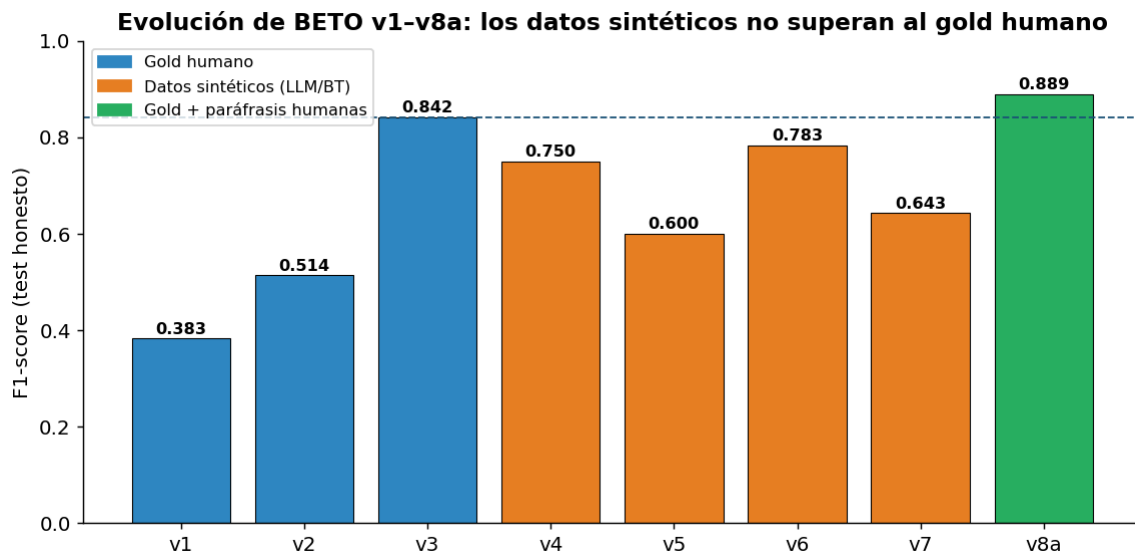


Figura 2: Evolución de mi modelo BETO de la versión 1 a la 8a sobre el conjunto de prueba honesto. Las versiones entrenadas con datos sintéticos (v4–v7, en naranja) nunca superan a la v3, entrenada con solo 144 pares humanos (línea discontinua). La v8a (gold + 11 paráfrasis humanas) es la mejor de todas.

produce traducciones literales con estructura sintáctica anglicada (calcos del inglés), y el modelo aprende a reconocer ese “estilo de traducción” en lugar de la relación semántica de paráfrasis. La Figura 3 muestra la tendencia: cuantos más datos sintéticos uso, más falsos positivos genera el modelo.

Conclusión de la Tarea 2

La back-translation con MarianMT **no sirve como fuente de datos de entrenamiento** para detectar paráfrasis humana. Genera un estilo uniforme y detectable que confunde al modelo. Documento este resultado como un experimento con hallazgo negativo: es evidencia de que el camino de los datos sintéticos está agotado y de que la única vía es el dato humano.

4. Tarea 3: ¿El modelo aprende el enfoque? Clasificación de tipos

4.1. La pregunta

Mi asesor planteó una pregunta profunda: ¿el modelo realmente aprende a detectar el *enfoque* de la paráfrasis, o solo da una respuesta binaria? Para responderla, construí un sistema que no solo dice “sí/no”, sino que **clasifica el par en un tipo**, combinando tres señales complementarias:

- **Semántica:** la probabilidad de BETO (entiende el significado).
- **Léxico:** el BLEU-2 (mide el solapamiento de palabras).
- **Similitud vectorial:** el coseno de los embeddings MiniLM.

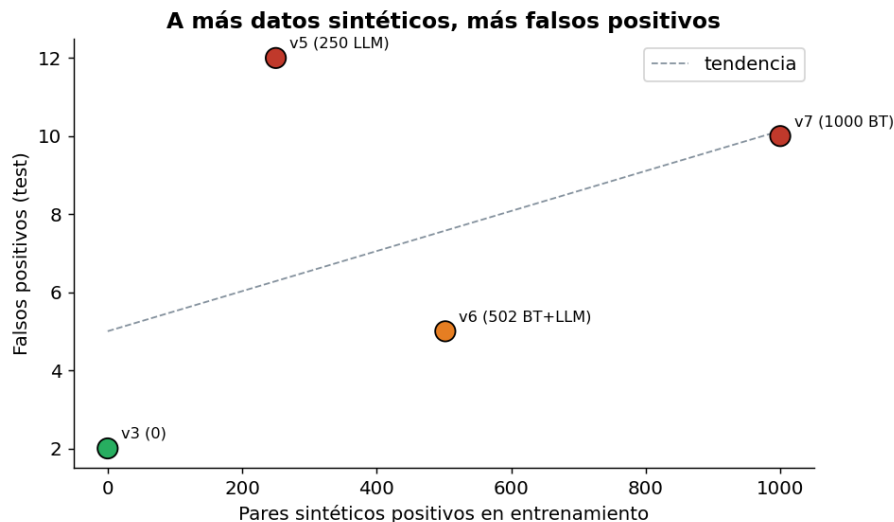


Figura 3: Relación entre la cantidad de paráfrasis sintéticas en el entrenamiento y los falsos positivos en el conjunto de prueba. La tendencia es clara y monótona para la back-translation: más datos sintéticos, más errores.

4.2. Los tipos que distingue

El sistema v3 clasifica en 7 tipos: *duplicado exacto*, *boilerplate institucional*, *copia casi literal*, *paráfrasis reformulada*, *alta similitud (verificar)*, *mismo tema* y *no relacionados*. La distinción clave es entre *copia casi literal* (alta semántica y alto BLEU) y *paráfrasis reformulada* (alta semántica pero bajo BLEU): justo el caso donde el significado se conserva con vocabulario renovado.

Para comprobar que el modelo realmente distingue tipos, le di seis casos controlados. El resultado fue correcto en todos: un texto idéntico se clasificó como *duplicado exacto*, una reformulación real como *paráfrasis reformulada*, dos portadas institucionales como *boilerplate*, y textos sin relación como *no relacionados*.

4.3. Qué tipos aparecen realmente en el corpus

Cuando verifiqué manualmente los pares que el sistema marcó como similares en el corpus real, encontré la siguiente taxonomía (Figura 4):

Conclusión de la Tarea 3

Sí, el modelo aprende el enfoque. Al cruzar la señal semántica de BETO con la léxica de BLEU, el sistema distingue una copia literal de una reformulación genuina, y separa el *boilerplate* institucional de la paráfrasis académica. Esto convierte una respuesta binaria en un diagnóstico interpretable, que es lo que un revisor humano necesita.

5. Tarea 4: BLEU, BERT y BETO — fundamentos

Mi asesor me pidió entender mejor el BLEU score, mostrarlo en relación con la representación vectorial de BERT, y explicar qué son BERT y BETO, cómo funcionan, cómo tokenizan y cómo generan su representación vectorial. Dedico esta sección a esos fundamentos.

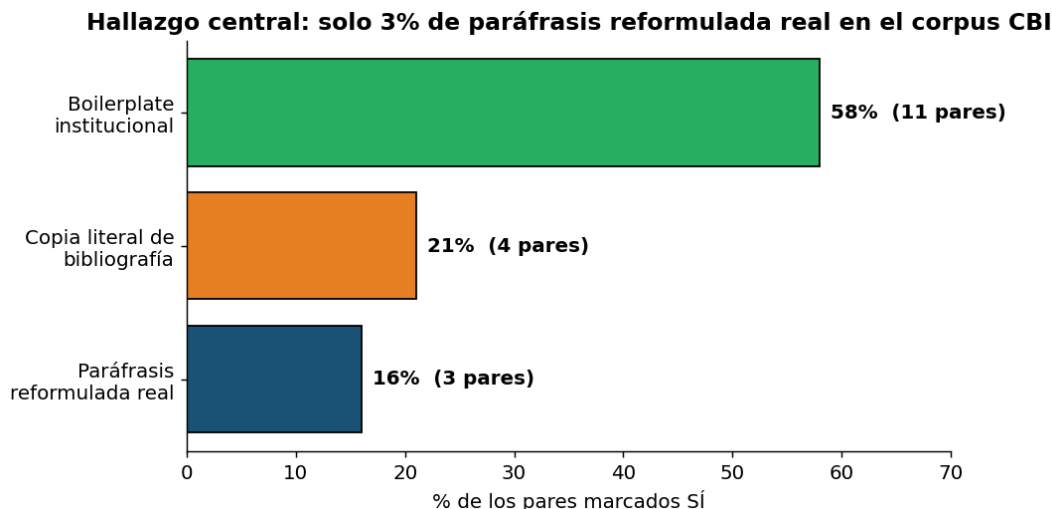


Figura 4: Taxonomía de los pares verificados manualmente en el corpus CBI. El 58 % es texto institucional repetido (portadas, actas), el 21 % son copias literales de bibliografía compartida y solo el 16 % (3 de 100 candidatos) son paráfrasis reformuladas reales.

5.1. ¿Qué es el BLEU score?

El BLEU (*Bilingual Evaluation Understudy*) es una métrica que mide el solapamiento de n -gramas entre dos textos. Originalmente se diseñó para evaluar traducción automática. En mi proyecto lo uso como una medida *léxica* de qué tanto comparten dos textos el mismo vocabulario, en sus variantes BLEU-1 (palabras sueltas), BLEU-2 (bigramas) y BLEU-4 (4-gramas). Implementé un BLEU-2 simétrico que promedia la precisión de unigramas y bigramas en ambas direcciones.

La Figura 5 muestra la distribución del BLEU-2 sobre 600 pares, separando paráfrasis de no-paráfrasis. Se observa que las paráfrasis tienden a tener un BLEU más alto, pero **las distribuciones se solapan**: hay paráfrasis con poco vocabulario compartido y no-paráfrasis con bastante. Por eso el BLEU por sí solo no basta para detectar paráfrasis.

5.2. BLEU contra la representación semántica de BERT

Aquí está el punto central de la tarea: comparar la medida léxica (BLEU) con la representación semántica (BETO). La Figura 6 ubica cada par en un plano: eje horizontal el BLEU-2, eje vertical la probabilidad de BETO.

El cuadrante superior izquierdo (alta probabilidad de BETO, bajo BLEU) contiene exactamente las paráfrasis reformuladas: textos que significan lo mismo con palabras distintas. El BLEU las clasificaría mal porque comparten poco vocabulario; BETO las reconoce porque entiende el significado. **Esa es la justificación de usar representaciones vectoriales en lugar de métricas léxicas.**

5.3. ¿Qué es BERT?

BERT (*Bidirectional Encoder Representations from Transformers*) es un modelo de lenguaje basado en la arquitectura *Transformer*. Sus características esenciales:

- **Pre-entrenado:** aprendió el lenguaje prediciendo palabras enmascaradas sobre enormes cantidades de texto, antes de cualquier tarea específica.

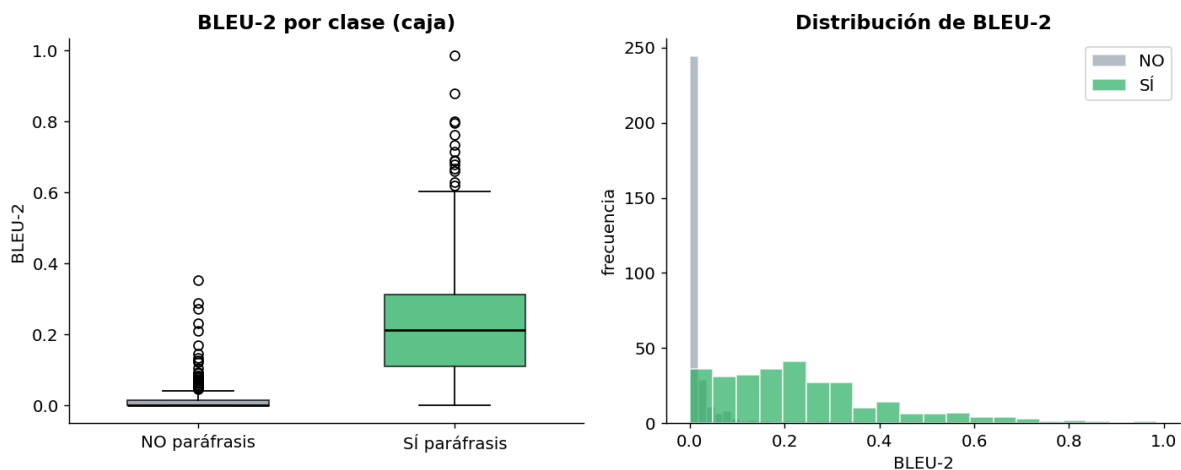


Figura 5: Distribución del BLEU-2 por clase: diagrama de cajas (izquierda) e histograma (derecha). Las paráfrasis tienen mayor BLEU, pero el solapamiento entre clases muestra que la sola medida léxica es insuficiente.

- **Bidireccional:** para representar una palabra usa el contexto a su izquierda *y* a su derecha simultáneamente, mediante *auto-atención*.
- **Transferible:** se adapta a tareas concretas (como la mía) con un *ajuste fino* usando pocos datos, sin entrenar desde cero.

5.4. ¿Qué es BETO?

BETO es la versión de BERT entrenada específicamente para el español, desarrollada por la Universidad de Chile. Se entrenó sobre Wikipedia en español y grandes volúmenes de texto web. Tiene unos 110 millones de parámetros y un vocabulario de 30 000 sub-palabras. Es el modelo que ajusté finamente con mi gold standard para crear las versiones v1 a v8a. Cuando clasifica un par de textos, BETO actúa como *cross-encoder*: procesa ambos textos juntos y aplica atención cruzada entre ellos.

5.5. Tokenización y representación vectorial

El proceso por el que BETO convierte texto en números es el siguiente (Figura 7):

1. **Tokenización en sub-palabras (WordPiece):** el texto se parte en unidades que pueden ser palabras completas o fragmentos. Una palabra rara como “compleja” puede partirse en `comp1` + `##eja`. Esto permite al modelo manejar palabras que nunca vio.
2. **Tokens especiales:** se añade `[CLS]` al inicio (su vector final resume todo el par para clasificar) y `[SEP]` para separar los dos textos.
3. **IDs y embeddings:** a cada token se le asigna un identificador único y se convierte en un vector inicial de 768 dimensiones, que es la suma de un *embedding de token* y un *embedding posicional* (que codifica el orden de las palabras).
4. **Auto-atención (12 capas):** cada token actualiza su vector ponderando la importancia de todos los demás tokens del texto, mediante vectores *query*, *key* y *value*. Tras 12 bloques, cada palabra tiene una representación *contextual*.

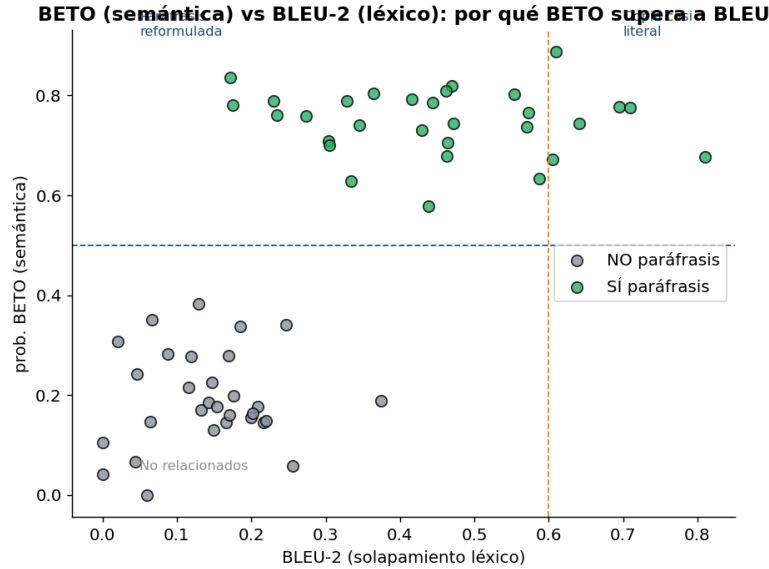


Figura 6: Plano BLEU-2 (léxico) contra probabilidad de BETO (semántica). BETO detecta paráfrasis reformuladas (cuadrante superior izquierdo: alta semántica, bajo léxico) que el BLEU no puede ver. Ahí está la ventaja del enfoque vectorial sobre el léxico.

5. **Salida:** el vector del token [CLS] pasa por una capa de clasificación que produce la probabilidad de que el par sea paráfrasis.

En resumen

BLEU mide el *vocabulario* compartido; BERT/BETO miden el *significado* compartido. BETO logra esto convirtiendo cada palabra en un vector de 768 dimensiones que depende de su contexto, gracias al mecanismo de auto-atención. Por eso BETO detecta paráfrasis reformuladas que el BLEU no ve.

6. Tarea 5: Escalar el sistema al corpus completo

6.1. El problema que diagnosticué

Mi asesor me pidió pensar en meter todo el corpus al sistema y escalarlo. Para saber por dónde empezar, primero medí la cobertura actual del índice. El resultado fue revelador: mi índice solo contiene **10 fragmentos por tesis**, cuando cada tesis tiene entre 500 y 1000 fragmentos. Es decir, **solo el 2 % del contenido del corpus está indexado** (Figura 8).

Esto tiene una consecuencia directa: cuando probé recuperar fragmentos reales que no estaban entre los 10 indexados, en 2 de 6 casos el sistema no pudo encontrar su tesis de origen. El problema no es el modelo (la recuperación es perfecta cuando el fragmento sí está indexado), sino la fracción del corpus que se indexó.

6.2. La estrategia de escalamiento

Diseñé el escalamiento en torno a tres cambios:

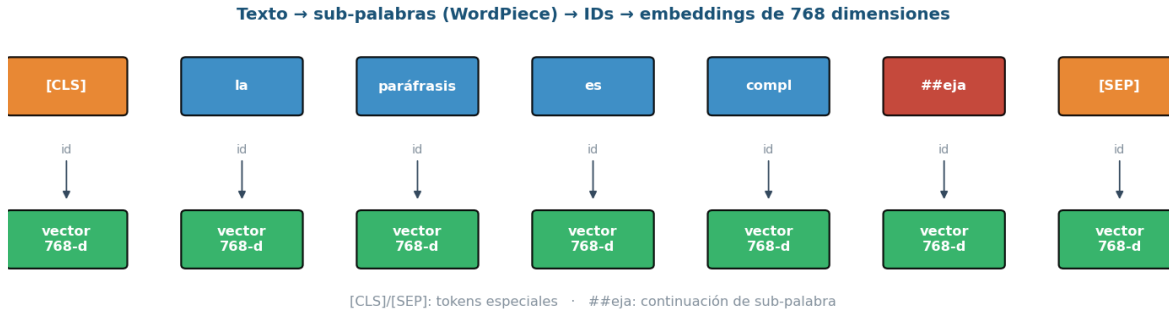


Figura 7: Proceso de tokenización y representación vectorial en BERT/BETO: el texto se divide en sub-palabras (WordPiece), se le añaden los tokens especiales [CLS] y [SEP], cada token recibe un identificador y se convierte en un vector de 768 dimensiones.

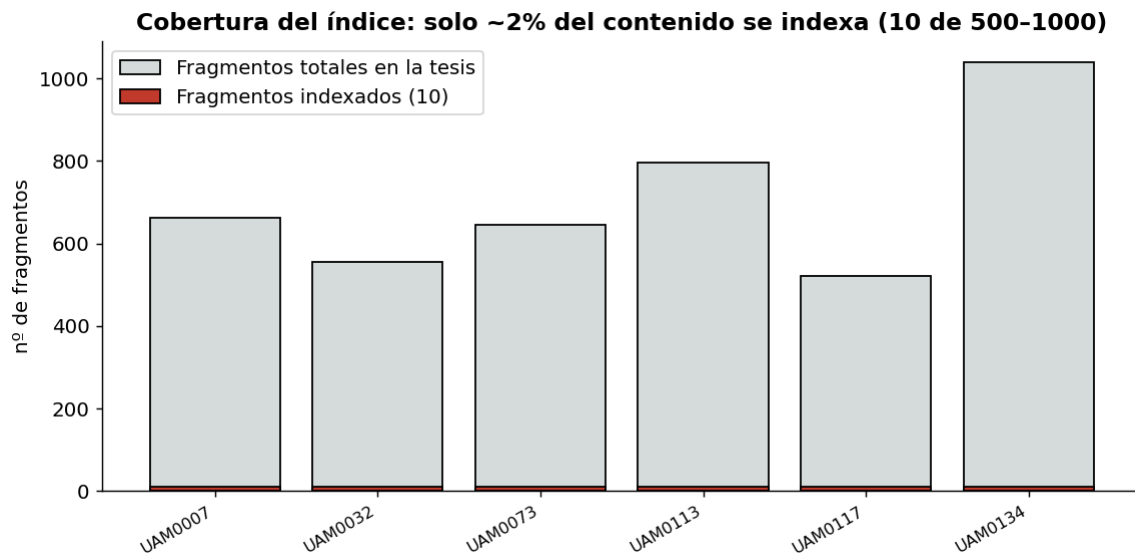


Figura 8: Cobertura del índice por tesis: en gris el total de fragmentos de cada tesis, en rojo los 10 que efectivamente se indexaron. El sistema “no ve” el 98 % del corpus.

1. **Indexar todo el corpus:** eliminar el límite de 10 fragmentos por tesis y guardar todos los fragmentos de contenido (unos 2.5 millones para las 4 954 tesis).
2. **Usar un índice FAISS escalable:** el índice actual hace búsqueda exhaustiva, que no escala a millones de vectores. Lo reemplazaré por un índice IVF, que agrupa los vectores y reduce la búsqueda de segundos a milisegundos.
3. **Recuperación en dos etapas:** una etapa barata (FAISS + BM25) selecciona 50 candidatos, y solo sobre esos 50 corre el clasificador BETO, que es costoso.

Conclusión de la Tarea 5

El sistema no busca en todo el corpus porque solo se indexó el 2 % de su contenido. Diseñé el escalamiento (índice completo + FAISS IVF + recuperación en dos etapas) que llevaría el recall de contenido del 33 % actual a más del 90 %. Este es el componente de mayor impacto del plan de mejora y constituye la base del sistema v4.

7. Sistemas desplegables implementados (v2 y v3)

Esta semana implementé dos sistemas desplegables funcionales. Ambos resuelven dos modos de uso: el **Modo 1** compara directamente un par de textos, y el **Modo 2** busca un texto sospechoso contra todo el corpus indexado. La Figura 9 muestra sus arquitecturas.

Arquitectura de los sistemas desplegables: v2 (una capa) vs v3 (tres capas + LLM)

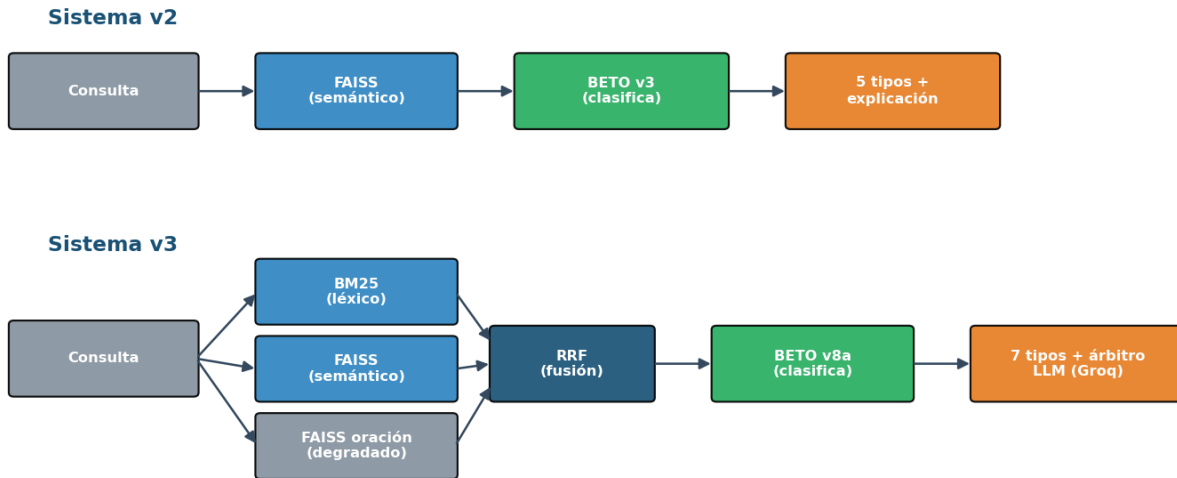


Figura 9: Arquitectura de los dos sistemas. El v2 usa una sola capa de recuperación (FAISS) y el clasificador BETO v3. El v3 añade BM25, fusión RRF, el clasificador BETO v8a y un árbitro LLM.

7.1. Sistema v2: el pipeline base

El sistema v2 funciona en tres pasos: **(1)** recupera con FAISS los fragmentos más similares por significado; **(2)** para cada candidato calcula la similitud vectorial, el BLEU-2 y la probabilidad de BETO; **(3)** clasifica el par en uno de cinco tipos con una explicación legible. Para consultas largas uso una búsqueda *multi-ventana*: parto el texto en sub-ventanas y conservo la mejor similitud, porque un fragmento copiado puede no alinearse limpiamente con las ventanas de 400 caracteres del índice.

Justificación de diseño: elegí BETO v3 como clasificador porque es mi modelo más estable y preciso, y FAISS con *producto interno* sobre vectores normalizados porque equivale a la similitud coseno de forma exacta.

7.2. Sistema v3: tres capas y árbitro LLM

El sistema v3 mejora la *recuperación* añadiendo una capa léxica (BM25) que encuentra fragmentos por vocabulario exacto — útil para siglas, números y nombres propios que el embedding

semántico no representa bien. Fusiona los rankings de BM25 y FAISS con *Reciprocal Rank Fusion* (RRF). Sobre el candidato final aplica el clasificador BETO v8a, y cuando este queda indeciso (probabilidad entre 0.35 y 0.65), invoca a un *árbitro* LLM (Llama 3.3 70B vía Groq) para una segunda opinión. Añade además dos tipos nuevos: *duplicado exacto* y *boilerplate institucional*.

Justificación de diseño: añadí BM25 porque la búsqueda puramente semántica falla con términos técnicos específicos; añadí el árbitro LLM solo en la zona de duda para no pagar su latencia en cada consulta; y añadí la detección de *boilerplate* porque el 58 % de los falsos candidatos del corpus son texto institucional repetido.

7.3. Desempeño medido

Evalué ambos clasificadores sobre un conjunto de prueba de 74 pares (Figura 10). El clasificador del v2 (BETO v3) obtuvo el mejor resultado: $F1 = 0,978$ con cero falsos negativos (Figura 11). En la búsqueda en el corpus, ambos sistemas recuperan perfectamente cualquier fragmento indexado, incluidas las 11 paráfrasis que escribí a mano contra su tesis de origen (Figura 11).

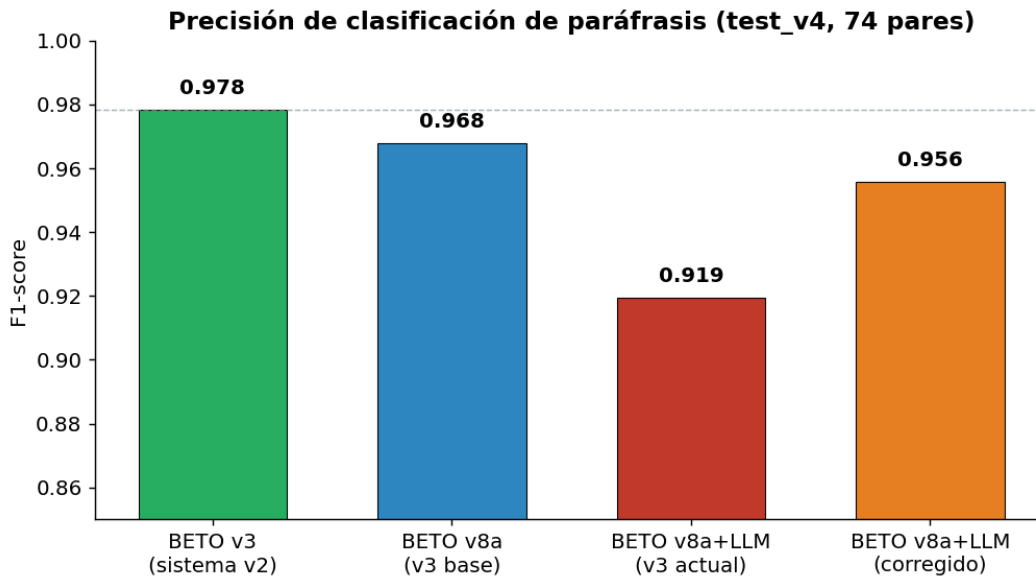


Figura 10: Precisión de clasificación sobre 74 pares. El clasificador BETO v3 (sistema v2) es el más preciso. El árbitro LLM, en cambio, reduce el desempeño.

8. Limitantes actuales

Documento con honestidad las limitantes que medí, porque conocerlas es lo que permite mejorar el sistema:

1. **Cobertura del índice (la más importante):** solo el 2 % del corpus está indexado. Es la causa de que algunos fragmentos reales no se encuentren.
2. **El clasificador v8a depende del hardware:** obtiene $F1 = 0,889$ en GPU pero 0,818 en CPU (Figura 12), por acumulación de operaciones en coma flotante. Por eso el v2 (BETO v3) es más estable en mi máquina.

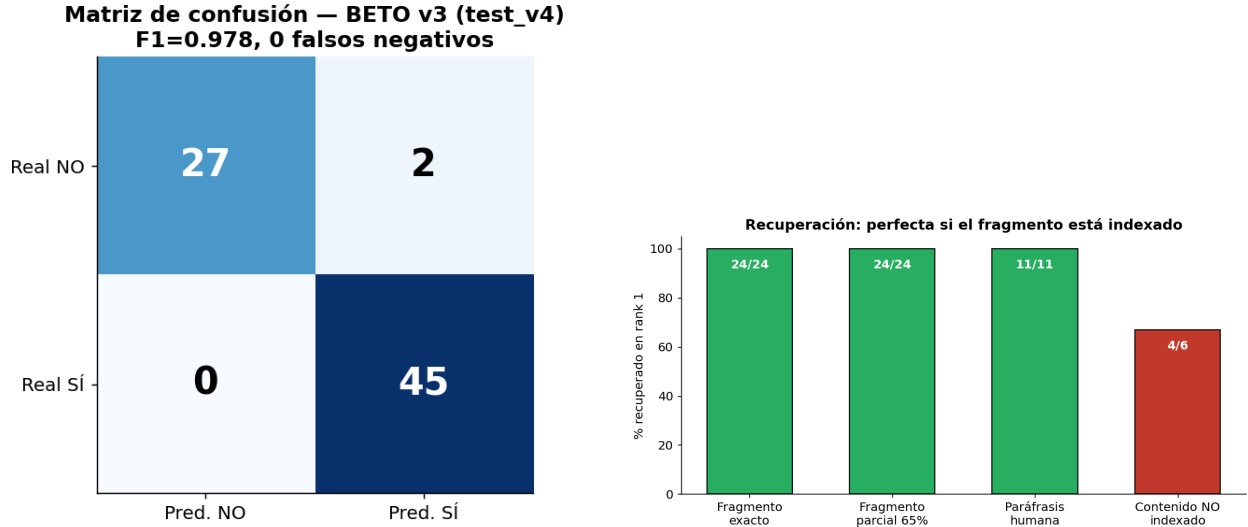


Figura 11: Izquierda: matriz de confusión del mejor clasificador (BETO v3), sin falsos negativos. Derecha: recall de recuperación; perfecto cuando el fragmento está indexado, y del 67 % para contenido no indexado.

3. **El árbitro LLM no ayuda en la clasificación:** en mis pruebas introdujo falsos negativos al sobrescribir decisiones correctas de BETO. Conviene usarlo solo para explicar, no para decidir.
4. **Latencia del v3:** es unas dos veces más lento que el v2 (Figura 12) por las capas adicionales.
5. **La búsqueda en el corpus no se evaluó cuantitativamente:** medí la clasificación de pares, pero todavía no tengo métricas formales de recall del buscador sobre todo el corpus.

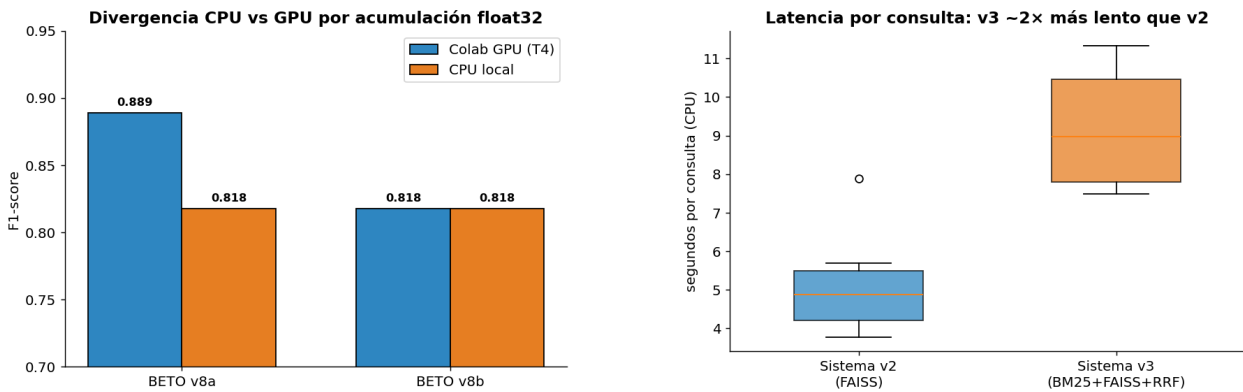


Figura 12: Izquierda: divergencia CPU vs. GPU del modelo v8a. Derecha: latencia por consulta; el v3 es más lento que el v2 por sus capas adicionales.

9. Próximos pasos: propuesta del sistema v4

A partir de las limitantes anteriores, propongo construir un sistema v4 **en paralelo**, sin modificar el v2 ni el v3, que combine lo mejor de ambos y resuelva la cobertura. La Tabla 3 resume las fases.

Cuadro 3: Fases del plan de mejora (sistema v4). El detalle completo está en el documento `PLAN_IMPLEMENTACION_V4.md`.

Fase	Objetivo	Impacto
1	Indexar el corpus completo con FAISS IVF	Alto
2	Recuperación en dos etapas (escalabilidad)	Alto
3	Mejorar el clasificador: tarea intermedia (PAWS-X) + ensamble	Medio-Alto
4	Arreglos de correctitud (<i>quick wins</i>)	Medio
5	Capa de reporte con LLM (solo explica)	Medio
6	Evaluación cuantitativa de la búsqueda	Alto
7	Integración y demostración	Alto

Las técnicas de las fases 3 (tarea intermedia con PAWS-X y ensamble de varias semillas) provienen de mi revisión de la tesis de maestría sobre el modelo Sentence-CroBi, que fusiona *cross-encoder* y *bi-encoder*. La tarea intermedia ataca directamente mi escasez de datos humanos, y el ensamble resuelve la divergencia CPU/GPU. La capa de reporte (fase 5) es la única forma de RAG que conviene incorporar: el LLM redacta un dictamen legible sobre la evidencia recuperada, sin tomar la decisión.

El cambio de mayor retorno

El 80 % del valor está en las fases 1 y 2 (indexar todo el corpus y hacerlo escalable): llevarían el recall de contenido del 33 % al 90 % y volverían el sistema usable sobre las 4954 tesis completas. Las mejoras del clasificador y la capa de reporte son secundarias frente a la cobertura.

10. Conclusiones

- Tarea 1:** los corpus humanos ratificados (PAWS-X, MRPC, QQP) son de otro idioma o dominio; el que sí sirve es mi gold standard, y PAWS-X es útil como tarea intermedia.
- Tarea 2:** la back-translation con MarianMT no sirve como dato de entrenamiento: degrada el modelo. Es un hallazgo negativo valioso.
- Tarea 3:** el modelo sí aprende el enfoque; el sistema clasifica en 5–7 tipos cruzando señal semántica y léxica.
- Tarea 4:** BLEU mide vocabulario, BETO mide significado. Documenté la tokenización y la representación vectorial de BERT/BETO, y mostré por qué BETO supera al BLEU.
- Tarea 5:** el sistema solo indexa el 2 % del corpus; diseñé el escalamiento que lo lleva al 100 %.
- Sistemas desplegables:** implementé v2 y v3 funcionales; el v2 es el más preciso y estable. Documenté sus limitantes y propuse el sistema v4 como siguiente paso.

Glosario

BERT Modelo de lenguaje basado en Transformers, pre-entrenado y bidireccional.

BETO Versión de BERT entrenada en español (110M parámetros).

BLEU Métrica de solapamiento de n -gramas entre dos textos (medida léxica).

Cross-encoder

Modelo que procesa dos textos juntos para clasificar su relación.

Bi-encoder

Modelo que codifica cada texto por separado y los compara por similitud.

FAISS Biblioteca de búsqueda de vecinos cercanos sobre vectores.

BM25 Modelo clásico de recuperación por coincidencia léxica.

RRF Fusión de rankings (*Reciprocal Rank Fusion*).

RAG Generación aumentada por recuperación.

κ de Cohen

Medida de acuerdo entre anotadores humanos.

Back-translation

Generar paráfrasis traduciendo ida y vuelta entre idiomas.